Give me list of problems in java, sql , javascript to workout daily and upload in github for 30-40 days, where it involves all interview based questions, concepts intermediate to pro, per day 2 hours approximately planning to spend on pratice questions, so give me list accordingly one by one

Below is a **35-day (5 weeks) daily practice plan** for **Java + SQL + JavaScript** designed for **~2 hours/day**, with **interview-style problems** from **intermediate → pro**, and a clean setup to **push to GitHub daily**.

GitHub setup (do this once, Day 0)

Repo name: `30-40-days-interview-practice`

Folders:

- `java/Day01/` … `java/Day35/`
- `sql/Day01/` … `sql/Day35/`
- `javascript/Day01/` … `javascript/Day35/`
- `notes/` (concept notes)
- `README.md`

Daily commit format: `Day 07 - Java BFS + SQL window + JS closures`

Each day deliverables

- Java: `Solution.java` (+ optional tests)
- SQL: `queries.sql`
- JS: `index.js`
- `notes/DayXX.md` (5–10 bullets: what you learned + gotchas)

Daily time split (2 hours)

- Java (DSA/Algo): **60 min**
- SQL (query patterns): **30 min**
- JavaScript (core + patterns): **30 min**

35-Day List (one by one)

Week 1 — Strong foundations + common patterns

Day 01

- **Java:** Two Sum + HashMap pattern (also return indices)
- **SQL:** SELECT basics + WHERE + ORDER BY + LIMIT (write 5 variations)
- **JS:** `var/let/const` , hoisting basics (write 6 small console examples)

Day 02

- **Java:** Valid Parentheses (stack)
- **SQL:** COUNT, DISTINCT, GROUP BY (top-N within groups not yet)
- **JS:** functions, scope, lexical environment (simple examples)

Day 03

- **Java:** Merge Two Sorted Lists (linked list)
- **SQL:** INNER JOIN / LEFT JOIN (3-table join once)
- **JS:** arrays essentials: map/filter/reduce (implement small tasks)

Day 04

- **Java:** Best Time to Buy and Sell Stock (Kadane-like / greedy)
- **SQL:** NULL handling: COALESCE, IS NULL; pitfalls
- **JS:** objects, destructuring, spread/rest (mini exercises)

Day 05

- **Java:** Binary Search (classic + first/last occurrence)
- **SQL:** HAVING vs WHERE (write 5 examples)
- **JS:** equality `==` vs `===` , truthy/falsy (edge cases)

Day 06

- **Java:** Reverse Linked List (iterative + recursive)
- **SQL:** Subqueries (IN, EXISTS) – write 3 each
- **JS:** closures (create counter, memo function)

Day 07

- **Java:** Longest Substring Without Repeating Characters (sliding window)
- **SQL:** Date functions + filtering by date ranges
- **JS:** callbacks vs promises (convert callback to promise)

Week 2 — Arrays/Strings mastery + SQL patterns

Day 08

- **Java:** 3Sum (sorting + two pointers)
- **SQL:** Top-N queries (LIMIT + ORDER BY) + ties discussion
- **JS:** promises chaining + error handling

Day 09

- **Java:** Container With Most Water (two pointers)
- **SQL:** Self join (e.g., employees-manager)
- **JS:** async/await rewrite of promise chain

Day 10

- **Java:** Group Anagrams (hashing + canonical key)
- **SQL:** UNION vs UNION ALL, dedupe pitfalls
- **JS:** event loop basics: microtask vs macrotask (demo)

Day 11

- **Java:** Product of Array Except Self (prefix/suffix)
- **SQL:** String functions (LIKE, CONCAT, SUBSTR) + pattern matching
- **JS:** prototype basics + class syntax difference

Day 12

- **Java:** Maximum Subarray (Kadane) + explain proof
- **SQL:** CASE WHEN (bucketing + conditional aggregates)
- **JS:** deep vs shallow copy (structuredClone vs JSON pitfalls)

Day 13

- **Java:** Find Minimum in Rotated Sorted Array
- **SQL:** Anti-join patterns (NOT EXISTS vs LEFT JOIN IS NULL)
- **JS:** write `debounce` and `throttle`

Day 14

- **Java:** Search in Rotated Sorted Array
- **SQL:** Dedup rows (ROW_NUMBER later; now DISTINCT and group)
- **JS:** implement small `memoize(fn)`

Week 3 — Trees/Graphs + SQL Window functions begin

Day 15

- **Java:** Binary Tree Level Order Traversal (BFS)
- **SQL:** Window intro: ROW_NUMBER() basics
- **JS:** DOM-less practice: build a mini state store (object + subscribe)

Day 16

- **Java:** Validate Binary Search Tree (DFS + bounds)
- **SQL:** RANK vs DENSE_RANK vs ROW_NUMBER
- **JS:** `this` binding: call/apply/bind (examples)

Day 17

- **Java:** Lowest Common Ancestor (BST or binary tree version)
- **SQL:** Running totals with SUM() OVER
- **JS:** implement `bind` polyfill (simple)

Day 18

- **Java:** Diameter of Binary Tree
- **SQL:** Moving averages / sliding windows (AVG OVER)
- **JS:** module patterns (CommonJS vs ES modules conceptually + example)

Day 19

- **Java:** Number of Islands (DFS/BFS)
- **SQL:** Window partitioning practice (per user/per category)
- **JS:** write a tiny `Promise.all` polyfill (simplified)

Day 20

- **Java:** Course Schedule (topological sort)
- **SQL:** "Top 1 per group" using ROW_NUMBER()
- **JS:** async pitfalls (sequential vs parallel awaits)

Day 21

- **Java:** Clone Graph (BFS + HashMap)
- **SQL:** Find gaps/islands in sequences (window technique)
- **JS:** implement `LRUCache` in JS (Map-based)

Week 4 — DP + harder SQL + JS interview classics

Day 22

- **Java:** Climbing Stairs (DP) + optimize to O(1)
- **SQL:** CTE basics (WITH) + readability refactor
- **JS:** recursion + tail call discussion; implement factorial safely

Day 23

- **Java:** House Robber (DP)
- **SQL:** Recursive CTE intro (simple hierarchy)
- **JS:** implement `flatten(array)` deep

Day 24

- **Java:** Coin Change (DP)
- **SQL:** Pivoting with CASE aggregates
- **JS:** implement `deepEqual(a,b)` (basic)

Day 25

- **Java:** Longest Increasing Subsequence (O(n log n) preferred)
- **SQL:** Query optimization mindset: indexes basics + sargability notes
- **JS:** time/space complexity explanation with examples

Day 26

- **Java:** Decode Ways (DP)
- **SQL:** Multi-CTE problem (build step-by-step)
- **JS:** implement `curry(fn)` (basic)

Day 27

- **Java:** Word Break (DP + HashSet)
- **SQL:** Window + conditional aggregates mixed problem
- **JS:** implement `once(fn)` + test

Day 28

- **Java:** Edit Distance (hard DP)
- **SQL:** Realistic report query: cohorts/retention style (mock tables ok)
- **JS:** implement `retry(fn, n)` with async

Week 5 — System-y problems + pro-level polishing

Day 29

- **Java:** Kth Largest Element (heap / quickselect)

- **SQL:** Transactions concept + isolation anomalies (write notes + examples)
- **JS:** memory leaks patterns + cleanup notes (timers, closures)

Day 30

- **Java:** Merge Intervals
- **SQL:** Upsert patterns (depends on DB; write generic patterns + notes)
- **JS:** implement `EventEmitter` (on/off/emit)

Day 31

- **Java:** Meeting Rooms II (min-heap)
- **SQL:** Star schema basics + join fact/dim style example
- **JS:** implement `asyncPool(limit, tasks)` (concurrency limiter)

Day 32

- **Java:** Implement Trie + search/prefix
- **SQL:** Complex join debugging checklist + rewrite messy query clean
- **JS:** implement `autocomplete` logic using Trie in JS (no UI needed)

Day 33

- **Java:** Design LRU Cache (HashMap + Doubly Linked List)
- **SQL:** Window-heavy challenge: “sessionization” (30 min inactivity)
- **JS:** implement `LRUCache` again but with tests + edge cases

Day 34

- **Java:** Serialize/Deserialize Binary Tree
- **SQL:** Interview pack: 5 short queries (joins, window, dedup, gaps, top-per-group)
- **JS:** “Frontend interview pack”: closures, this, promises, event loop mini quiz (code)

Day 35

- **Java:** Mock interview day: pick 1 random from past and solve faster + explain aloud (write explanation)
- **SQL:** Mock interview day: 1 tough window query + optimize readability
- **JS:** Mock interview day: implement 1 utility from scratch (debounce/curry/promiseAll)

If you want 40 days (add these 5)

Day 36: Java — Median of Two Sorted Arrays (hard) | SQL — recursive hierarchy | JS — parser-like string tokenizer

Day 37: Java — Minimum Window Substring | SQL — data quality checks query set | JS — virtual DOM concept notes

Day 38: Java — Regex Matching (DP) | SQL — performance anti-patterns | JS — implement `compose/pipe`

Day 39: Java — Union-Find (Accounts Merge) | SQL — slowly changing dimensions notes | JS — custom `fetch` wrapper with timeout

Day 40: Java — System design-lite coding (rate limiter) | SQL — audit log query | JS — rate limiter implementation

---

If you tell me which SQL dialect you're using (**MySQL / Postgres / SQL Server / Oracle**), I'll tweak the SQL tasks to match exact syntax and give you **daily problem statements + sample tables** so you can paste and run immediately.

Give me beginner to intermediate level questions for 10 days in all 3 laungages

Perfect 👍 Here's a **10-day beginner → intermediate plan** for **Java + SQL + JavaScript**, designed for **~2 hours per day**.

Time split suggestion (daily):

- Java – 60 mins
- SQL – 30 mins
- JavaScript – 30 mins

Each day includes **clear problem statements** (no solutions so you can practice & push to GitHub).

✅ Day 1 – Basics & Logic Building

☕ Java

1. Reverse a string.
2. Check if a number is a palindrome.
3. Find the largest of three numbers.
4. Count vowels and consonants in a string.

🗄️ SQL

1. Select all columns from a table `employees`.
2. Retrieve employees with salary > 50000.
3. Sort employees by salary descending.
4. Count total number of employees.

🌐 JavaScript

1. Reverse a string.
2. Check if a number is even or odd.
3. Write a function to find factorial of a number.
4. Print numbers from 1–100 (FizzBuzz).

✅ Day 2 – Arrays & Filtering

☕ Java

1. Find the maximum and minimum in an array.
2. Remove duplicates from an array.
3. Find second largest element.
4. Sum of all elements in array.

🗄️ SQL

1. Get distinct department names.
2. Count employees per department.
3. Retrieve employees hired after 2022.
4. Use `LIKE` to find names starting with 'A'.

🌐 JavaScript

1. Remove duplicates from array.
2. Use `map()` to square numbers.
3. Use `filter()` to return even numbers.
4. Use `reduce()` to sum array elements.

✅ Day 3 – Strings & Grouping

☕ Java

1. Check if two strings are anagrams.
2. Count character frequency in string.
3. Find first non-repeating character.
4. Remove all whitespaces from string.

🗄️ SQL

1. GROUP BY department and show average salary.
2. Find highest salary.
3. Find employees with NULL manager_id.
4. Use `CASE WHEN` to categorize salary (Low/Medium/High).

🌐 JavaScript

1. Count character frequency in string.
2. Check palindrome string.
3. Flatten a nested array (1 level).
4. Convert array of strings to uppercase.

✅ Day 4 – Searching & Sorting

☕ Java

1. Implement linear search.
2. Implement binary search.
3. Sort array using bubble sort.
4. Check if array is sorted.

🗄️ SQL

1. Top 5 highest paid employees.
2. Find duplicate records using GROUP BY.
3. Difference between WHERE and HAVING (write examples).
4. INNER JOIN employees and departments.

🌐 JavaScript

1. Sort numbers ascending & descending.
2. Find index of element manually.

3. Remove falsy values from array.
4. Merge two arrays.

✅ Day 5 – Hashing Concepts

☕ Java

1. Two Sum problem.
2. Find duplicates using HashSet.
3. Count frequency using HashMap.
4. Intersection of two arrays.

🗄️ SQL

1. Find second highest salary.
2. LEFT JOIN example.
3. Find employees without department.
4. Use subquery to get employees above average salary.

🌐 JavaScript

1. Two Sum.
2. Count occurrences in array.
3. Group elements by property.
4. Convert object to array and vice versa.

✅ Day 6 – Stack & Patterns

☕ Java

1. Valid parentheses using Stack.
2. Reverse a stack.
3. Check balanced brackets.
4. Implement simple stack using array.

🗄️ SQL

1. Use ROW_NUMBER().
2. Get top salary per department.
3. Use RANK() vs DENSE_RANK().
4. Running total using SUM() OVER().

🌐 JavaScript

1. Implement stack class.
2. Valid parentheses check.
3. Implement queue.
4. Reverse words in sentence.

✅ Day 7 – Linked List Basics

☕ Java

1. Reverse linked list.
2. Detect cycle (Floyd's algorithm).
3. Find middle node.
4. Merge two sorted linked lists.

🗄️ SQL

1. Self join (employee-manager relationship).
2. Find employees earning more than their manager.
3. Use EXISTS.
4. Delete duplicate rows.

🌐 JavaScript

1. Implement linked list.
2. Reverse linked list.
3. Find middle node.
4. Remove duplicates from linked list.

✅ Day 8 – Recursion & Intermediate Logic

☕ Java

1. Fibonacci using recursion & DP.
2. Climbing stairs.
3. Power of number (x^n).
4. Generate all subsets (basic version).

🗄️ SQL

1. CTE (WITH clause).

2. Recursive CTE (simple hierarchy).
3. Moving average salary.
4. Partition by department.

🌐 JavaScript

1. Fibonacci recursion.
2. Flatten deep nested array.
3. Deep clone object.
4. Implement debounce function.

✅ Day 9 – Trees & Data Handling

☕ Java

1. Inorder traversal of binary tree.
2. Level order traversal.
3. Check if tree is symmetric.
4. Max depth of tree.

🗄️ SQL

1. Find gaps in sequence.
2. Monthly sales aggregation.
3. Pivot using CASE.
4. Find consecutive login days.

🌐 JavaScript

1. Implement simple binary tree.
2. BFS traversal.
3. Implement throttle.
4. Promise chaining example.

✅ Day 10 – Interview Simulation Day

☕ Java

1. Longest substring without repeating characters.
2. Merge intervals.
3. Kth largest element.

4. LRU Cache (basic version).

🗄️ SQL

1. Top 3 salary per department.
2. Sessionization (30-min inactivity rule conceptually).
3. Cohort analysis basic query.
4. Optimize a slow query (explain indexing idea).

🌐 JavaScript

1. Implement Promise.all().
2. Implement curry function.
3. Event loop explanation with example.
4. Build mini LRU cache.

📌 How to Push to GitHub Daily

Structure:

```
Day01/
  java/
  sql/
  javascript/
  README.md
```

Inside README:

- Problems solved
- Time taken
- What you learned
- Difficulties faced

If you want, I can next:

- ✅ Give sample table schema for SQL practice
- ✅ Provide 10-day solution set
- ✅ Convert this into a printable PDF roadmap
- ✅ Create a LeetCode-style tracking checklist

Tell me what you prefer 👍

Give sample table schema for SQL practice

✅ Provide 10-day solution set

✅ Convert this into a printable PDF roadmap

✅ Create a LeetCode-style tracking checklist

one by one

Thought for 21s >

1) Sample table schema for SQL practice (with seed data)

Use this file to create tables + insert data for joins, window functions, streaks, pivots, etc.:

Download SQL schema + seed data

Tables included

- `departments`
- `employees` (manager relationship + nullable department)
- `salaries` (history for window functions)
- `sales` (monthly agg + pivot)
- `logins` (consecutive days + sessionization)

---

2) 10-day solution set (Java + SQL + JavaScript)

GitHub-ready folder structure: `Day01` … `Day10` with:

- `java/Solution.java`
- `sql/queries.sql`
- `javascript/index.js`

ChatGPT 5.3 Instant

Download 10-day solution set (ZIP)

---

3) Printable PDF roadmap (10 days)

Download PDF roadmap

---

4) LeetCode-style tracking checklist

Pick either format:

- 10_day_tracking_checklist.xlsx

  Checklist

- Download checklist (Markdown)